Construção de Serviços

Nome do Processo

Construção de Serviços

Autores: Pedro Custódio, Alexandre Pierri, Lucas Roberto, Gabriel Albertini e Kawan Mark

Data de emissão: 29/04/2026

Revisor: Equipe FINTA

Data de revisão: 29/04/2026

Objetivo do Documento

Este documento descreve a construção de dois serviços do sistema FINTA em uma arquitetura orientada a serviços: o **Serviço de Autenticação** e o **Serviço de Consulta de Cotação**. A entrega apresenta o estilo de coordenação adotado, o fluxo de interação entre os serviços e as estratégias de teste unitário e de integração, com evidências da execução dos testes.

Estilo de Coordenação dos Serviços

O estilo de coordenação adotado é **orquestração síncrona**.

A escolha se justifica porque o backend controla explicitamente a sequência do fluxo. Para consultar uma cotação, o cliente primeiro precisa estar autenticado; o endpoint de cotação valida o token Bearer JWT e somente depois aciona o serviço responsável por consultar preço, cache e provedor externo. Assim, a ordem das chamadas é centralizada pelo Worker/API, e não distribuída por eventos independentes entre serviços.

Serviços e componentes envolvidos:

- **Serviço de Autenticação**: realiza login, valida credenciais e emite token Bearer JWT.
- **Serviço de Consulta de Cotação**: recebe a solicitação autenticada e retorna cotação de ação ou criptomoeda.
- **Cache de Cotação**: mantém snapshots de cotações em KV para reduzir chamadas externas.
- **Provedores Externos**: Brapi para ações e CoinCap para criptomoedas.

Fluxo principal:

1. O usuário informa e-mail e senha no frontend web.
2. O frontend envia `POST /auth/login` ao Serviço de Autenticação.
3. O Serviço de Autenticação consulta o banco D1, valida a senha e emite um Bearer JWT.
4. O frontend envia `GET /ativos/{ticker}` com `Authorization: Bearer <token>`.
5. O endpoint de cotação valida o token antes de consultar o ativo.
6. O Serviço de Consulta de Cotação verifica o cache KV.
7. Se houver cache válido, retorna a cotação em cache.
8. Se o cache estiver ausente, chama o provedor externo, salva o snapshot e retorna a cotação ao cliente.
9. Se o cache estiver expirado, retorna o snapshot stale imediatamente e agenda atualização em background.

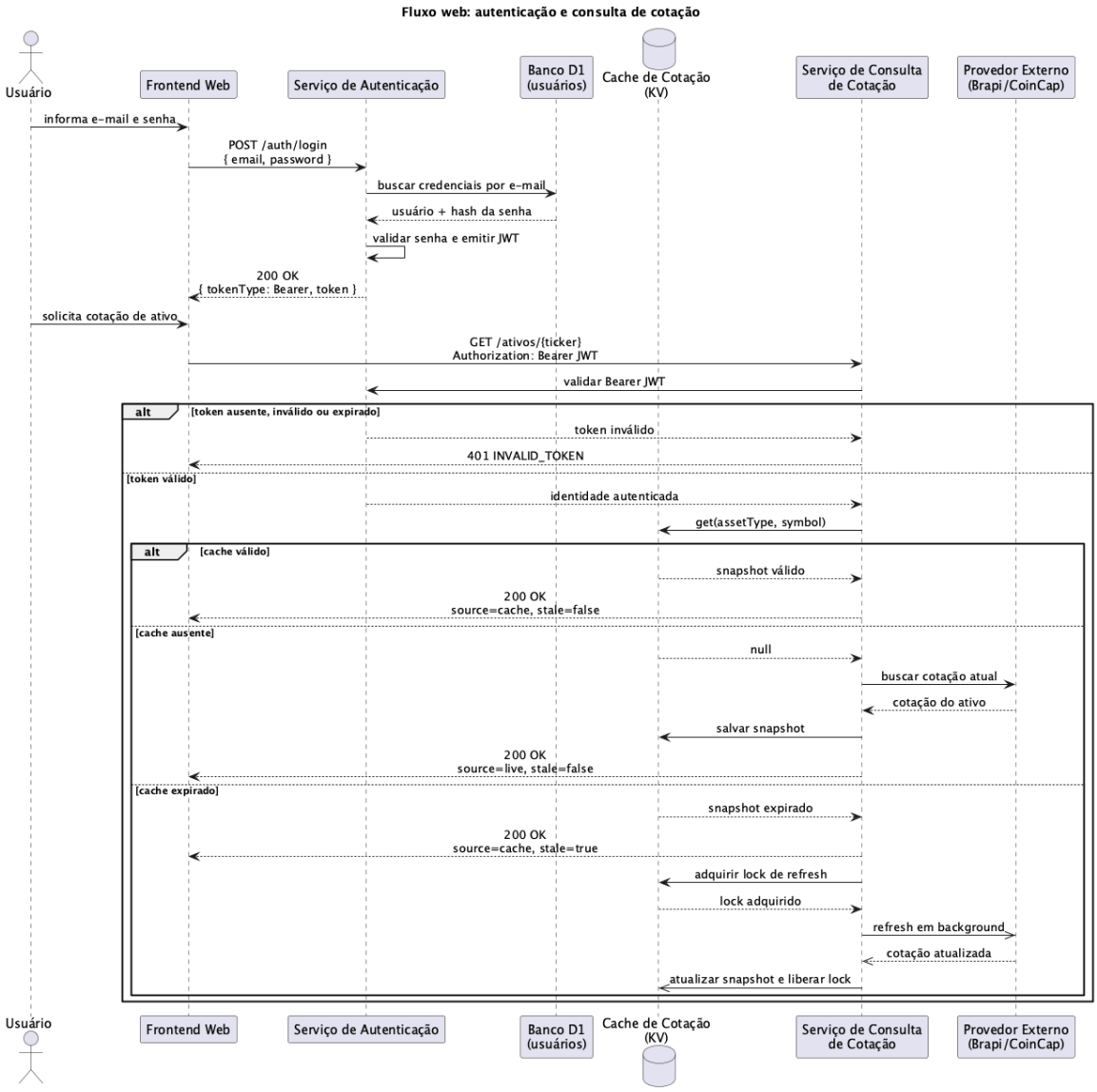

Figure 1: Diagrama de sequência do fluxo de autenticação e cotação

Estratégia de Teste

A estratégia de testes foi dividida entre teste unitário e teste de integração. O teste unitário foca a lógica interna do Serviço de Consulta de Cotação. O teste de integração valida a comunicação entre Autenticação e Consulta de Cotação no backend.

Tipo de Teste: Teste Unitário

Serviço testado: Serviço de Consulta de Cotação (`@finta/price-query`).

Objetivo do Teste: validar a lógica interna de consulta de cotação, principalmente a decisão entre buscar dados no cache ou no provedor externo. Os cenários cobrem cache frio, cache quente, cache expirado, atualização em background, ausência de cotação cacheada e busca por prefixo.

Técnica: testes automatizados com Vitest, usando dublês/mocks para o gateway de dados de mercado e uma implementação em memória do repositório de cache. Dessa forma, a regra de negócio é validada isoladamente, sem depender de Cloudflare KV ou de provedores externos reais.

Arquivo de teste: `packages/price-query/src/application/price-query-service.test.ts`.

Critério de Finalização: todos os testes unitários do pacote `@finta/price-query` devem passar, incluindo os cenários de cache e atualização.

Comando executado:

```
pnpm --filter @finta/price-query test
```

Tipo de Teste: Teste de Integração

Serviços testados: Serviço de Autenticação e Serviço de Consulta de Cotação.

Objetivo do Teste: validar que a consulta de cotação só é executada mediante autenticação válida e que o fluxo integrado retorna cotação com metadados de cache. Também são verificados casos negativos, como token ausente, inválido ou expirado.

Técnica: testes automatizados com Vitest sobre o Worker HTTP (`apps/backend-cloudflare`). O teste aciona endpoints reais da aplicação em ambiente de teste, com banco D1 falso, cache KV falso e provedores externos mockados por `fetch`. Isso valida a integração entre autenticação, autorização, consulta de cotação e persistência em cache.

Arquivo de teste: `apps/backend-cloudflare/src/index.test.ts`.

Critério de Finalização: todos os testes do backend devem passar, incluindo os cenários de login/autenticação, consulta de cotação autenticada, uso de cache e respostas `401` para tokens inválidos.

Comando executado:

```
pnpm --filter backend-cloudflare test
```

Evidências dos testes realizados (prints)

Abaixo estão os prints gerados com a ferramenta Freeze a partir da execução real dos testes no terminal.

Evidência 1 — Teste unitário do Serviço de Consulta de Cotação

Evidência 2 — Teste de integração do backend

```
> @finta/price-query@0.0.0 test /Users/pedro/Faculdade/topicos-avancados-eng-software/projeto-finta/packages/price-query
> vitest run


 RUN  v3.2.4 /Users/pedro/Faculdade/topicos-avancados-eng-software/projeto-finta/packages/price-query


 ❯ src/application/price-query-service.test.ts [queued]

 Test Files 0 passed (1)
      Tests 0 passed (0)
   Start at 22:11:57
   Duration 104ms
 ✓ src/application/price-query-service.test.ts (6 tests) 4ms
   ✓ PriceQueryService > returns a live quote on cold cache 2ms
   ✓ PriceQueryService > returns a cached quote on warm cache 0ms
   ✓ PriceQueryService > marks cached quotes as stale once ttl expires 0ms
   ✓ PriceQueryService > refreshes stale quotes in the background 0ms
   ✓ PriceQueryService > returns null when cached quote is missing 0ms
   ✓ PriceQueryService > searches cached quotes by asset type and limit 0ms

 Test Files  1 passed (1)
      Tests  6 passed (6)
   Start at  22:11:57
   Duration  223ms (transform 28ms, setup 0ms, collect 27ms, tests 4ms, environment 0ms, prepare 36ms)
```

Figure 2: Execução do teste unitário do pacote price-query

Telefone: (011) 4353-2900 Fax (011) 4109-5994

Curso de Ciências da Computação

TPM_Fase de Construção

```
> backend-cloudflare@ test /Users/pedro/Faculdade/topicos-avancados-eng-software/projeto-finta/apps/backend-cloudflare
> vitest run


 RUN  v3.2.4 /Users/pedro/Faculdade/topicos-avancados-eng-software/projeto-finta/apps/backend-cloudflare


 ❯ src/lib/password.test.ts [queued]

 Test Files 0 passed (7)
      Tests 0 passed (0)
   Start at 22:11:58
   Duration 103ms
 ✓ src/lib/api-key.test.ts (4 tests) 2ms
 ✓ src/lib/jwt.test.ts (4 tests) 4ms
 ✓ src/adapters/price-query/brapi-market-data-gateway.test.ts (5 tests) 5ms
 ✓ src/adapters/price-query/coincap-market-data-gateway.test.ts (6 tests) 5ms
 ✓ src/lib/auth.test.ts (5 tests) 6ms

 ❯ src/index.test.ts [queued]
 ❯ src/lib/password.test.ts 1/2

 Test Files 5 passed (7)
      Tests 25 passed (26)
   Start at 22:11:58
   Duration 202ms
 ✓ src/lib/password.test.ts (2 tests) 50ms

 ❯ src/index.test.ts 0/68

 Test Files 6 passed (7)
      Tests 26 passed (94)
   Start at 22:11:58
   Duration 402ms

 ❯ src/index.test.ts 1/68

 Test Files 6 passed (7)
      Tests 27 passed (94)
   Start at 22:11:58
   Duration 503ms
 ✓ src/index.test.ts (68 tests) 131ms

 Test Files  7 passed (7)
      Tests  94 passed (94)
   Start at  22:11:58
   Duration  572ms (transform 304ms, setup 0ms, collect 547ms, tests 204ms, environment 1ms, prepare 297ms)
```

Figure 3: Execução do teste de integração do backend